COMPUTER ENGINEERING

QUEZON CITY

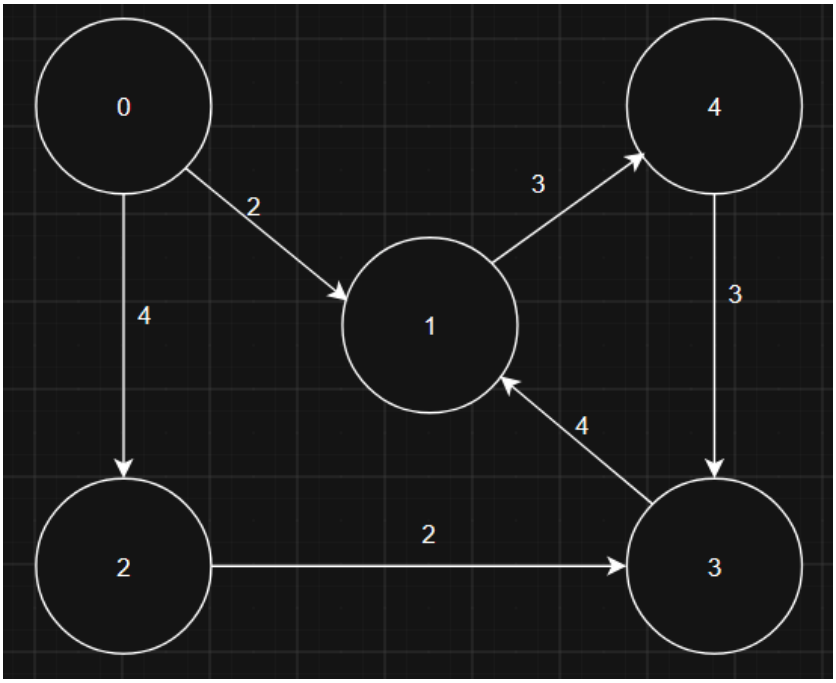
| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithm | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 02/10/2026 | **DATE SUBMITTED:** | 02/10/2026 |
| **NAME:** | Realubit, Seth Andrew I. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 10.1** **Graphs**

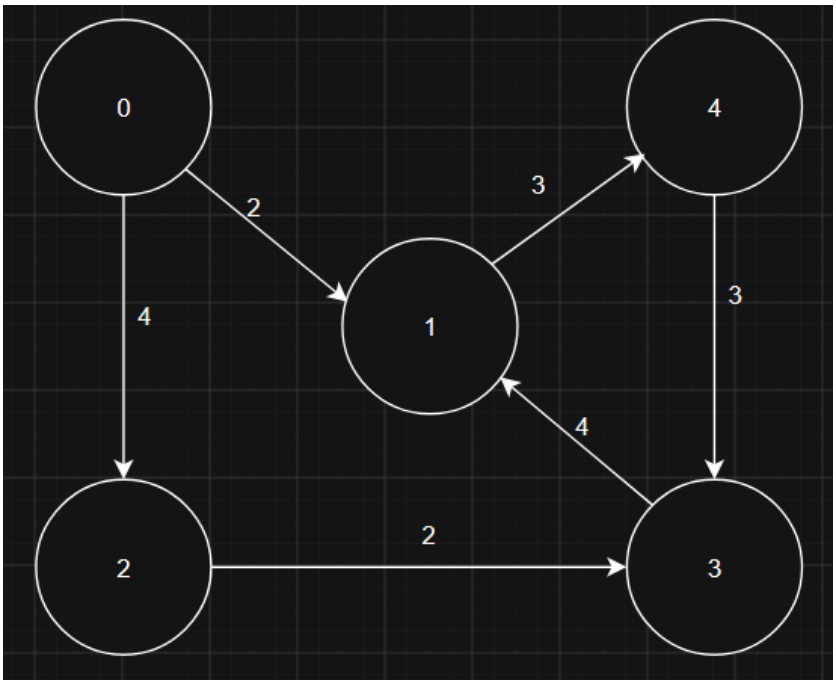
**1. PROCEDURE OUTPUT**

**ILO A**

**ILO A.1 Create a Graph**




**ILO A.2 Adjacency Matrix**




| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 0 | 2 | 4 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 3 | 0 |
| 2 | 0 | 0 | 0 | 2 | 0 | 0 |
| 3 | 0 | 4 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 3 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 0 |

**ILO A.3 Adjacency List**

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

**C++ Syntax of ILO A**

CPE010_HOA3p1_REALUBIT_sc1.cpp | CPE010_HOA3p1_REALUBIT_sc3.cpp | CPE010_HOA3p1_REALUBIT

HOA10p1 > ILO A > CPE010_HOA3p1_REALUBIT_sc1.cpp > display_AdjMatrix(adjNode *, int)

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
    private:
    // insert new nodes into adjacency list from given graph
        adjNode* getAdjListNode(int value, int weight, adjNode* head) {
            adjNode* newNode = new adjNode;
            newNode->val = value;
            newNode->cost = weight;
            newNode->next = head; // point new node to current head
            return newNode;
        }
        int N; // number of nodes in the graph
    public:
        adjNode **head; //adjacency list as array of pointers
        // Constructor
        DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i){
            head[i] = nullptr;
        // construct directed graph by adding edges to it
            for (unsigned i = 0; i < n; i++) {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;
                // insert in the beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
                // point head pointer to new node
                head[start_ver] = newNode;
            }
        }
```

```
45      }
46
47          // Destructor
48          ~DiaGraph() {
49              for (int i = 0; i < N; i++){
50                  delete[] head[i];
51              }
52              delete[] head;
53          }
54  };
55
56  // print all adjacent vertices of given vertex
57  void display_AdjList(adjNode* ptr, int i){
58      while (ptr != nullptr) {
59          std::cout << "(" << i << ", " << ptr->val
60              << ", " << ptr->cost << ") ";
61          ptr = ptr->next;
62      }
63      std::cout << std::endl;
64  }
65
66  void display_AdjMatrix(adjNode* ptr, int i){
67      const int total_vertices = 6;
68      int matrixRow[total_vertices] = {0};
69
70      while (ptr != nullptr) {
71          matrixRow[ptr->val] = ptr->cost;
72          ptr = ptr->next;
73      }
74
75      std::cout << i << " ";
76      for (int j = 0; j < total_vertices; j++) {
77          std::cout << matrixRow[j] << " ";
78      }
79      std::cout << std::endl;
80  }
```

```cpp
// graph implementation
int main(){
    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list \n" << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    std::cout << "\n  0 1 2 3 4 5\n";
    for(int i=0; i < N; i++) {
        display_AdjMatrix(diagraph.head[i], i);
    }
    return 0;
}
```

In ILO A, the process was a mix of being tedious and not, the program itself had some errors that needed to be indented, forgotten brackets, missing semi-colons and the missing syntax for implementing the adjacency matrix. In creating the graphs, I mainly based it from the output from the terminal, afterwards I searched the definition of the weight and finally learned the actual use, I learned that it was the example of the cost in moving from one node to another.

## ILO B.1 Depth First-Search

HOA10p1 > ILO B > DFS > CPE010_HOA3p1_REALUBIT_sc3.cpp > main()

```cpp
// ILO B.1 - Depth-First Search (stack-based, edge list graph)

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

```

```cpp
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

## ILO B.2 Breadth-First Search

HOA10p1 > ILO B > BFS > CPE010_HOA3p1_REALUBIT_sc2.cpp > operator<<<T>(std::ost

```cpp
// ILO B.2 - Breadth-First Search (queue-based, edge list graph)

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

```

```
86  private:
87      size_t V;
88      std::vector<Edge<T>> edge_list;
89  };
90
91  template <typename T>
92  auto create_reference_graph()
93  {
94      Graph<T> G(9);
95
96      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
97      edges[1] = {{2, 2}, {5, 3}};
98      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
99      edges[3] = {{4, 2}, {7, 3}};
100     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
101     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
102     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
103     edges[7] = {{3, 3}, {6, 4}};
104     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
105
106     for (auto &i : edges)
107         for (auto &j : i.second)
108             G.add_edge(Edge<T>{i.first, j.first, j.second});
109
110     return G;
111 }
112
```

```
113 template <typename T>
114 auto breadth_first_search(const Graph<T> &G, size_t dest)
115 {
116     std::queue<size_t> queue;
117     std::vector<size_t> visit_order;
118     std::set<size_t> visited;
119
120     queue.push(1); // Assume that BFS always starts from vertex ID 1
121
122     while (!queue.empty())
123     {
124         auto current_vertex = queue.front();
125         queue.pop();
126
127         // If the current vertex hasn't been visited in the past
128         if (visited.find(current_vertex) == visited.end())
129         {
130             visited.insert(current_vertex);
131             visit_order.push_back(current_vertex);
132
133             for (auto e : G.outgoing_edges(current_vertex))
134                 queue.push(e.dest);
135         }
136     }
137     return visit_order;
138 }
139
```

```cpp
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

From my observation there were generally three differences, firstly, the DFS (Depth-First Search) was essentially using stacks while the BFS (Breadth-First Search) was using queues. Then, DFS only pushes elements into the stack if that node is unvisited and the BFS pushes everything. Essentially, DFS acts as a filter to remove the already visited neighbors while the BFS pushes every neighbor while relying on the visited to check for duplicates. Lastly, because of their nature the DFS and BFS have different results, while they all start at the 1 as their vertex, they then stray far from each other with DFS moving on to 5 while BFS heading to 2.

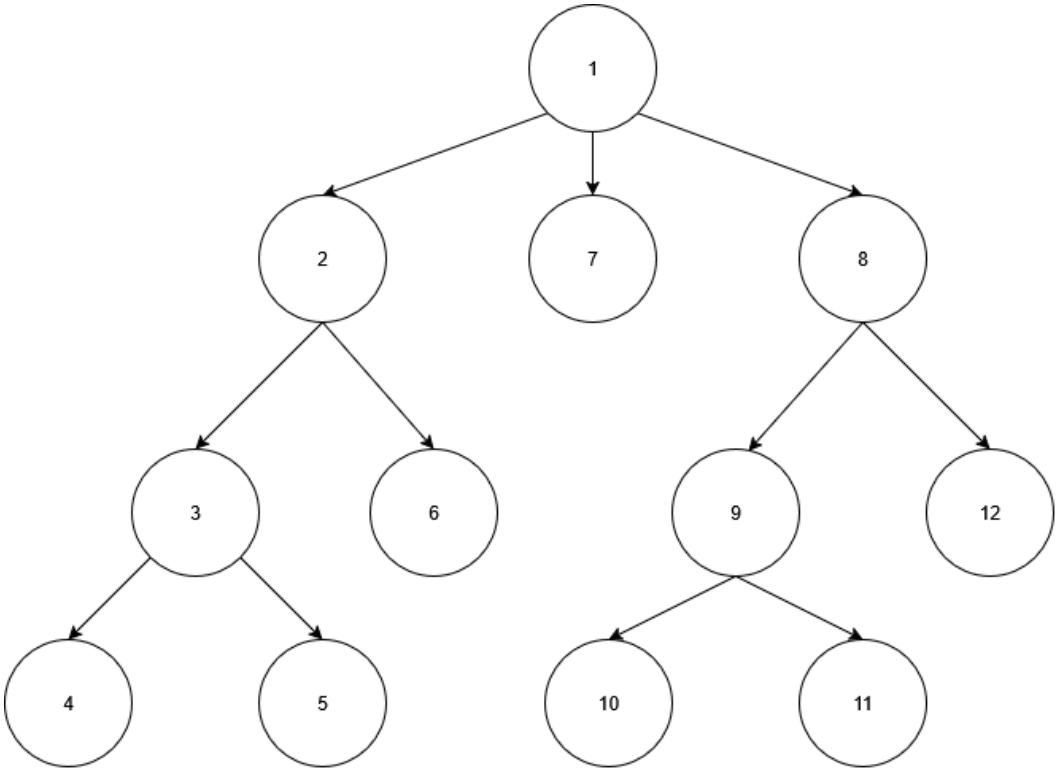
## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The algorithm that fits or would be the most helpful is the DFS or Depth-First Search. Basing off from the traversal, which follows a path as deep as possible before visiting other nodes, it fits the description of DFS especially with the backtracking.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.




| | Visit Order | Direction |
|---|---|---|
| **DFS** | 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 | Down first, then back up |
| **BFS** | 1, 2, 7, 8, 3, 6, 9, 12, 4, 5, 10, 11 | Across each level, then down |

| Pseudocode |
| --- |
| **dfs(v):**<br>**MARK v visited**<br>**for each unvisited u**<br>**adjacent to v:**<br>**dfs(u)**<br><br>**preorder(node):**<br>**if node is null: return**<br>**VISIT(node)**<br>**for each child c of node:**<br>**preorder(c)** |

**Code Implementation**

```
template <typename T>
struct Node{
    int val:
    std::vector<Node*> children;
};

template <typename T>
void preOrder(Node<T>* node){
    if (!node) return;
    std::cout<< node->val << " ";
    for (Node* c : node->children){
        preOrder(c);
    }
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
In the breadth first search, the data structure that was used to implement it was a queue. It is first-in, first-out (FIFO), so vertices that are processed earlier, which gives the level-by-level order.

4. How many times can a node be visited in the BFS?

The amount of times a node can be visited in BFS is exactly once. The visited set guarantees that each vertex is processed and added to the visit order only for a single time.

## Conclusion

In this hands-on activity we delved into a new topic, to be specific, it was graphs. In a way it could be understood or mistaken as what seemed like a tree, but that was because graphs are trees but not the other way around. Apart from that the lecture also consisted of two searches such as breadth-first search (BFS) and depth-first search (DFS). In a way it is almost similar to the tree traversal such as the preOrder, inOrder and postOrder. The

only thing is there is another type of traversal such as level order that the BFS utilizes, it basically visits every node in what seems like a tree data structure level by level. Meanwhile, the DFS utilizes the preOrder traversal such as pushing the left node as much as possible before returning and to the other nodes. The lecture also came with weight, or essentially cost from moving one node to another.

During the procedure activity, the majority of the code was already given and the only thing left was reusing the code and implementing another. Specifically, implementing the graph with the right weight of moving from one node to another and the implementation of the adjacent matrix which had different methods but chose to stick with the simplest of just implementing it inside the main. Moving on to ILO B where there were multiple errors inside the already made program and only needed to change some minor changes like where there was another template inside the template leading to the error and eventually being solved with using a different letter. Along with the missing return so the program runs properly.

Overall, the activity was lenient, despite the errors that were made deliberately they were not too difficult to debug and only needed some minor changes due to the majority of the code already in the discussion itself. Apart from that during the supplementary activity the implementation of the traversal was not entirely difficult because following an own made algorithm or pseudocode would make it easier, this along with the previous experience of the preOrder implementation during trees.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Claude AI**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Explanation of concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I mainly used Claude by asking questions to deepen my understanding of the procedure because while the implementation was already done, there was no specific explanation of why and how it was being used to actually implement the program.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY